

ТЕМА 5

Параметричний поліморфізм. Шаблони

Питання для самостійного опрацювання

Виконав: Вівчар Вадим Вікторович

Група: АЛК-43

Дисципліна: Об'єктно-орієнтоване програмування

У звіті наведено відповіді на контрольні запитання з теми «Параметричний поліморфізм. Шаблони». Матеріал охоплює поняття узагальненого програмування, шаблонних класів, інстанціювання, параметрів шаблонів, особливостей статичних членів, успадкування шаблонів, а також використання класів string і vector зі стандартної бібліотеки C++.

1. Пояснити зміст термінів «узагальнене програмування», «параметризований клас».

Узагальнене програмування - це підхід, за якого дані й алгоритми описуються так, щоб їх можна було застосовувати до різних типів даних без зміни самого опису. Параметризований клас, або шаблонний клас, - це клас, у якому тип даних задається як параметр. Такий клас слугує шаблоном для побудови конкретних класів під час підстановки фактичного типу.

2. Наведіть приклад опису шаблонного класу Точка_в_N-вимірному просторі.

Опис можна подати так:

```
template
class PointN {
    T coord[N];
public:
    T& operator[](int i) { return coord[i]; }
    const T& operator[](int i) const { return coord[i]; }
};
```

У наведеному прикладі T задає тип координат, а N - кількість вимірів простору.

3. Які типи можна використовувати як фактичний параметр-тип?

Як фактичні параметри шаблону можна використовувати вбудовані типи, користувацькі класи, інші шаблони, константні вирази, адреси об'єктів або зовнішніх функцій, а також неперевантажені вказівники на члени класу. За однакового набору аргументів генерується один і той самий тип.

4. Чи існують обмеження на класи, які можуть бути використанні як фактичні параметри при конкретному інстанціюванні шаблону?

Так. Шаблон залежить лише від тих властивостей фактичного типу, які явно використовуються в його реалізації. Тому тип повинен підтримувати потрібні операції. Наприклад, якщо в шаблоні створюються масиви елементів, викликаються оператори присвоювання, порівняння або копіювання, то конкретний клас має це дозволяти.

5. Чим відрізняється використання звичайного класу і інстанційованого? Шаблонного і нешаблонного класу?

Звичайний клас задається без параметрів і використовується безпосередньо. Інстанційований клас - це конкретна версія шаблону, отримана після підстановки фактичного типу, наприклад Stack або Stack. Шаблонний клас є

лише загальним описом, а нешаблонний - уже готовим класом для безпосереднього створення об'єктів.

6. В яких випадках статичні члени шаблонного класу дублюються для кожного інстанціювання?

Статичні члени шаблонного класу є специфічними для кожної конкретної інстанціації. Це означає, що для `X` і `X` будуть створені різні статичні змінні. Отже, дублювання відбувається щоразу, коли шаблон інстанціюється іншим набором параметрів.

7. Чи може шаблон бути похідним від звичайного класу? Від шаблону? В яких випадках від шаблонного може походити звичайний клас?

Так. Похідний шаблон класу може походити як від звичайного класу, так і від іншого шаблону. Звичайний клас може бути похідним від шаблонного лише тоді, коли шаблон попередньо інстанційовано конкретним типом, наприклад `class D : public C { ... };`

8. На якому етапі виконання програми створюється виконуваний код для версії шаблону класу? Для функції - члена шаблону?

Інстанціювання шаблону відбувається на етапі компіляції. Саме тоді компілятор генерує конкретну версію класу або функції для фактичних параметрів. Функції-члени шаблонного класу також генеруються під час компіляції, коли виникає потреба в їх використанні.

9. В чому полягають перевага і недоліки використання об'єктів типу `string`?

Переваги `string` полягають у зручній роботі з рядками: доступні оператори присвоювання, конкатенації, порівняння, індексації, введення та виведення. Крім того, розмір рядка може динамічно змінюватися. Недоліком є дещо більші накладні витрати порівняно зі звичайними масивами символів, а також те, що для дуже низькорівневих операцій інколи доводиться перетворювати `string` у рядок стилу `C` через `c_str()`.

10. Шаблонний клас `string` і його методи для роботи з рядками символів.

Клас `string` є спеціалізацією шаблонного класу `basic_string`. Він призначений для роботи з однобайтовими символьними рядками. Серед поширених операторів і методів: `=`, `+`, `+=`, `==`, `!=`, `<`, `>`, `[]`, `assign()`, `insert()`, `erase()`, `find()`, `append()`, `clear()`, `copy()`, `c_str()`, `length()`, `size()`, `capacity()`, `max_size()`, `resize()`. Ці засоби забезпечують гнучке створення, зміну, порівняння та аналіз рядків.

11. Шаблонний клас `vector` і його методи для роботи з динамічними векторами.

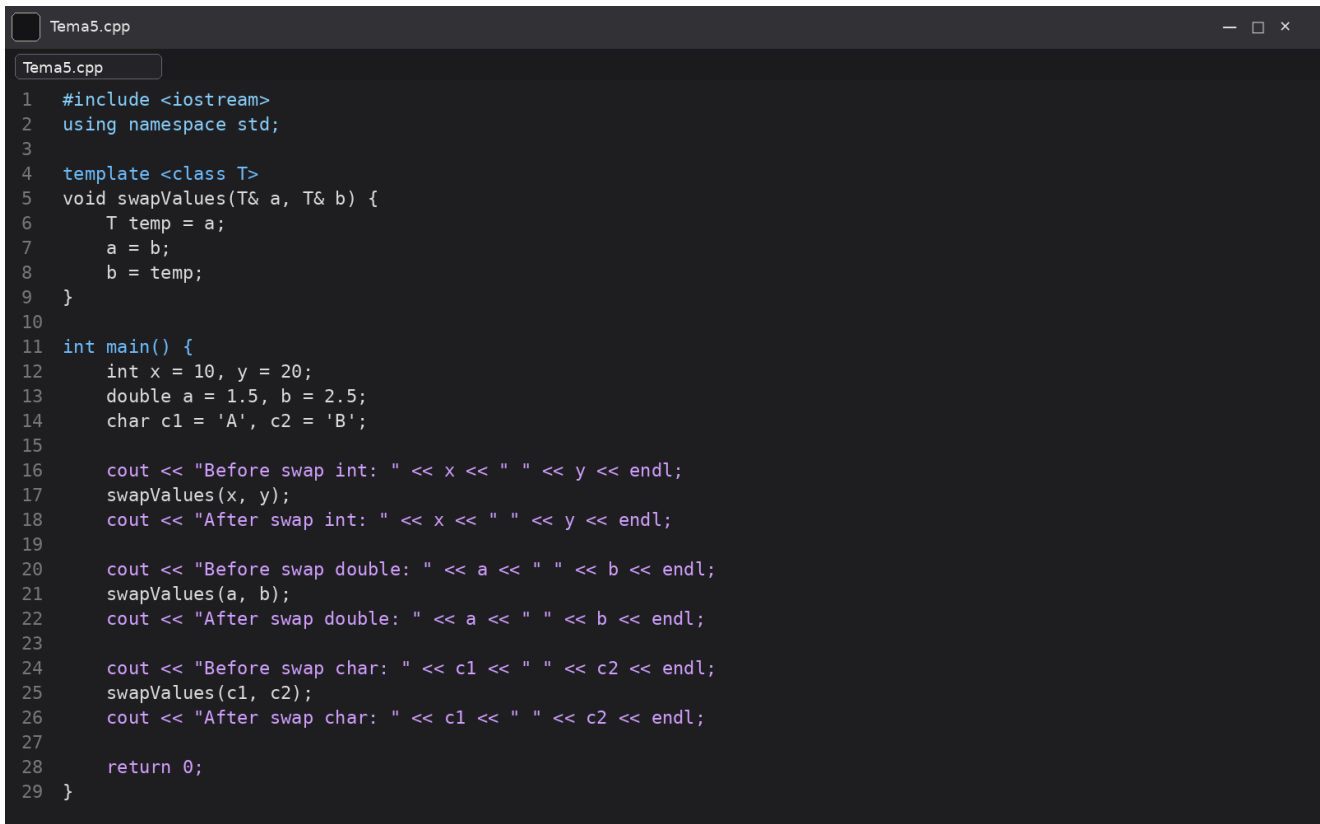
Клас `vector` - це шаблонний контейнер динамічного масиву, який підтримує довільний доступ до елементів. Він дає змогу зберігати об'єкти будь-якого типу та змінювати кількість елементів у процесі роботи програми. Основні методи: `assign()`, `at()`, `back()`, `begin()`, `capacity()`, `clear()`, `empty()`, `end()`, `erase()`, `front()`, `insert()`, `max_size()`, `pop_back()`, `push_back()`, `reserve()`, `resize()`, `size()`, `swap()`. Перевагою `vector` є швидкий доступ за індексом і зручне розширення контейнера.

Висновок

Шаблони в C++ є основним засобом реалізації параметричного поліморфізму та узагальненого програмування. Вони дозволяють створювати універсальні класи й функції, які працюють із різними типами даних без дублювання коду. Класи `string` і `vector` демонструють практичну цінність шаблонів у стандартній бібліотеці C++ та широко застосовуються під час розробки програм.

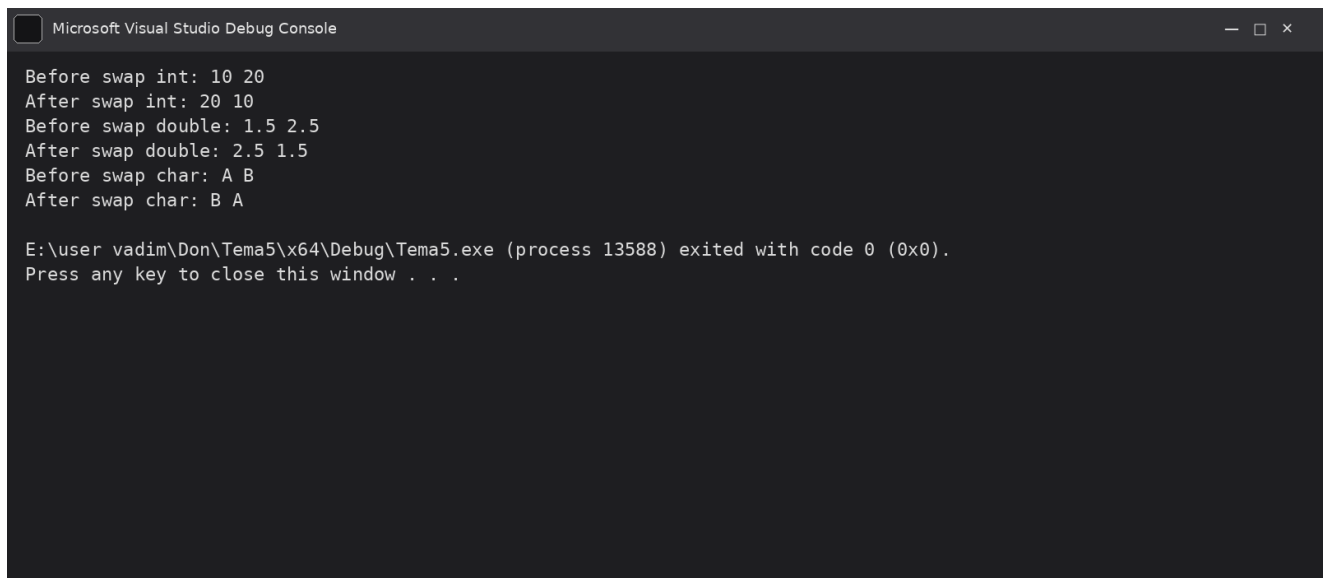
Практичні приклади у Visual Studio

Нижче наведено практичні приклади, виконані у середовищі Microsoft Visual Studio, які підтверджують основні положення теми «Параметричний поліморфізм. Шаблони».



```
Tema5.cpp
1  #include <iostream>
2  using namespace std;
3
4  template <class T>
5  void swapValues(T& a, T& b) {
6      T temp = a;
7      a = b;
8      b = temp;
9  }
10
11 int main() {
12     int x = 10, y = 20;
13     double a = 1.5, b = 2.5;
14     char c1 = 'A', c2 = 'B';
15
16     cout << "Before swap int: " << x << " " << y << endl;
17     swapValues(x, y);
18     cout << "After swap int: " << x << " " << y << endl;
19
20     cout << "Before swap double: " << a << " " << b << endl;
21     swapValues(a, b);
22     cout << "After swap double: " << a << " " << b << endl;
23
24     cout << "Before swap char: " << c1 << " " << c2 << endl;
25     swapValues(c1, c2);
26     cout << "After swap char: " << c1 << " " << c2 << endl;
27
28     return 0;
29 }
```

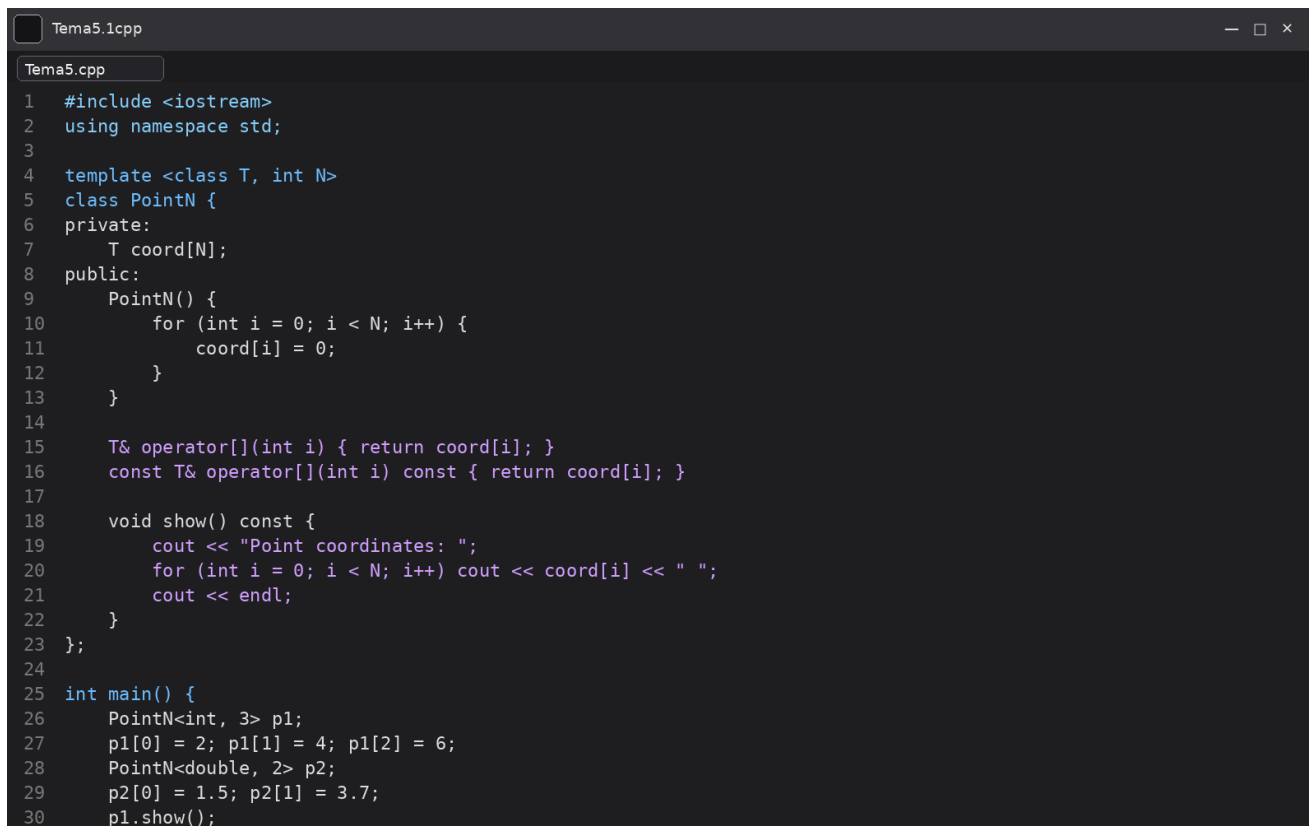
Рисунок 1 – Код програми з демонстрацією шаблонної функції *swapValues*

The image shows a screenshot of the Microsoft Visual Studio Debug Console window. The window has a dark background and a title bar that reads "Microsoft Visual Studio Debug Console". The output text is as follows:

```
Before swap int: 10 20
After swap int: 20 10
Before swap double: 1.5 2.5
After swap double: 2.5 1.5
Before swap char: A B
After swap char: B A

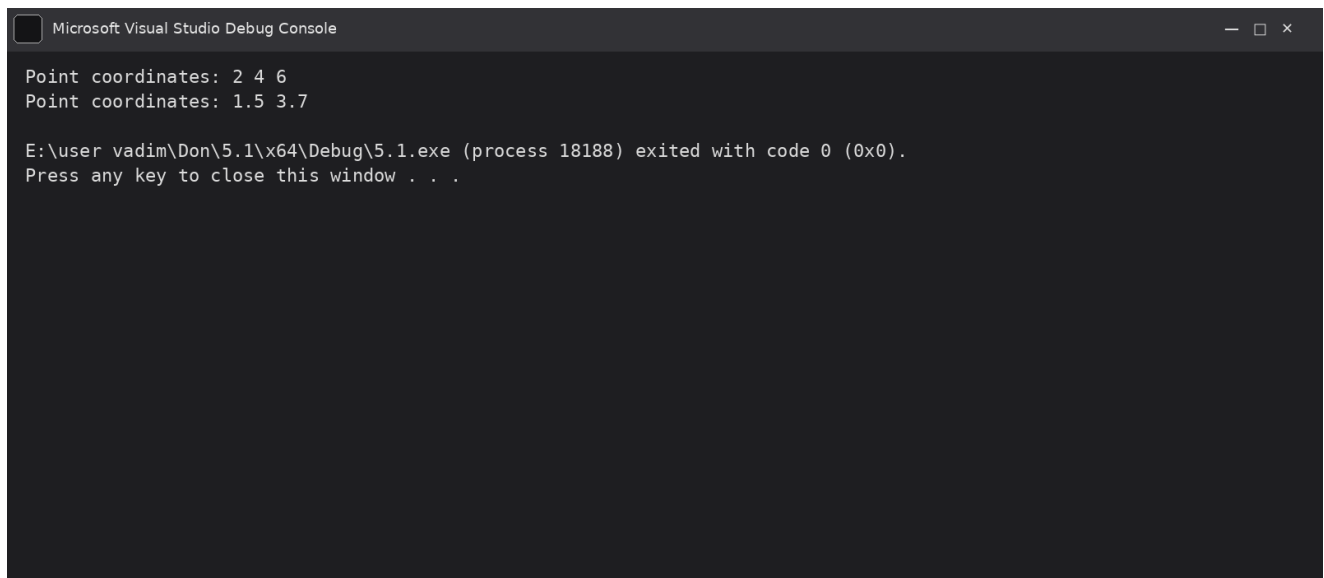
E:\user vadim\Don\Tema5\x64\Debug\Tema5.exe (process 13588) exited with code 0 (0x0).
Press any key to close this window . . .
```

Рисунок 2 – Результат виконання програми з шаблонною функцією swapValues



```
1  #include <iostream>
2  using namespace std;
3
4  template <class T, int N>
5  class PointN {
6  private:
7      T coord[N];
8  public:
9      PointN() {
10         for (int i = 0; i < N; i++) {
11             coord[i] = 0;
12         }
13     }
14
15     T& operator[](int i) { return coord[i]; }
16     const T& operator[](int i) const { return coord[i]; }
17
18     void show() const {
19         cout << "Point coordinates: ";
20         for (int i = 0; i < N; i++) cout << coord[i] << " ";
21         cout << endl;
22     }
23 };
24
25 int main() {
26     PointN<int, 3> p1;
27     p1[0] = 2; p1[1] = 4; p1[2] = 6;
28     PointN<double, 2> p2;
29     p2[0] = 1.5; p2[1] = 3.7;
30     p1.show();
```

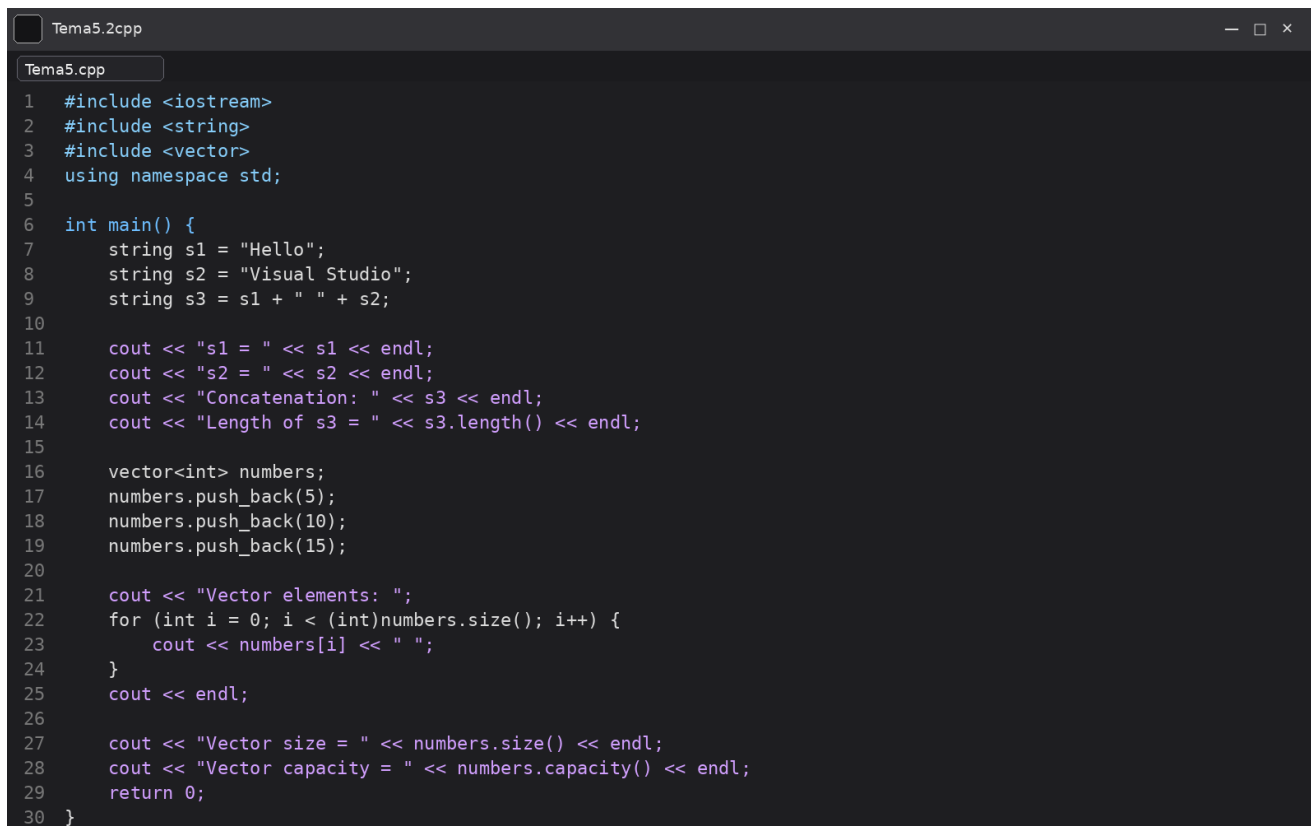
Рисунок 3 – Код програми з демонстрацією шаблонного класу *PointN*



The image shows a screenshot of the Microsoft Visual Studio Debug Console. The window has a title bar with the text "Microsoft Visual Studio Debug Console" and standard window control buttons (minimize, maximize, close). The console output is as follows:

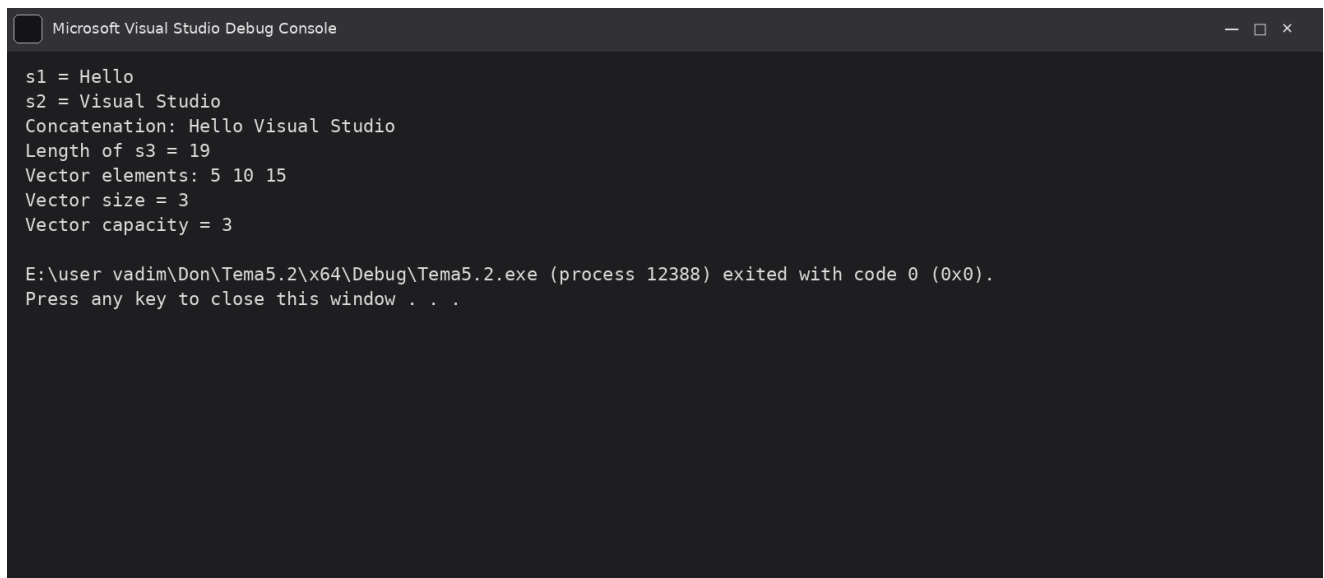
```
Point coordinates: 2 4 6  
Point coordinates: 1.5 3.7  
  
E:\user vadim\Don\5.1\x64\Debug\5.1.exe (process 18188) exited with code 0 (0x0).  
Press any key to close this window . . .
```

Рисунок 4 – Результат виконання програми для шаблонного класу PointN



```
1  #include <iostream>
2  #include <string>
3  #include <vector>
4  using namespace std;
5
6  int main() {
7      string s1 = "Hello";
8      string s2 = "Visual Studio";
9      string s3 = s1 + " " + s2;
10
11     cout << "s1 = " << s1 << endl;
12     cout << "s2 = " << s2 << endl;
13     cout << "Concatenation: " << s3 << endl;
14     cout << "Length of s3 = " << s3.length() << endl;
15
16     vector<int> numbers;
17     numbers.push_back(5);
18     numbers.push_back(10);
19     numbers.push_back(15);
20
21     cout << "Vector elements: ";
22     for (int i = 0; i < (int)numbers.size(); i++) {
23         cout << numbers[i] << " ";
24     }
25     cout << endl;
26
27     cout << "Vector size = " << numbers.size() << endl;
28     cout << "Vector capacity = " << numbers.capacity() << endl;
29     return 0;
30 }
```

Рисунок 5 – Код програми з демонстрацією роботи шаблонних класів *string* і *vector*



```
Microsoft Visual Studio Debug Console

s1 = Hello
s2 = Visual Studio
Concatenation: Hello Visual Studio
Length of s3 = 19
Vector elements: 5 10 15
Vector size = 3
Vector capacity = 3

E:\user vadim\Don\Tema5.2\x64\Debug\Tema5.2.exe (process 12388) exited with code 0 (0x0).
Press any key to close this window . . .
```

Рисунок 6 – Результат виконання програми для класів string і vector